



Introduction to Database Systems Spring 2025, Lecture 8: Indexes, Min/Max Summaries & Join Algorithms

Intended Learning Outcomes

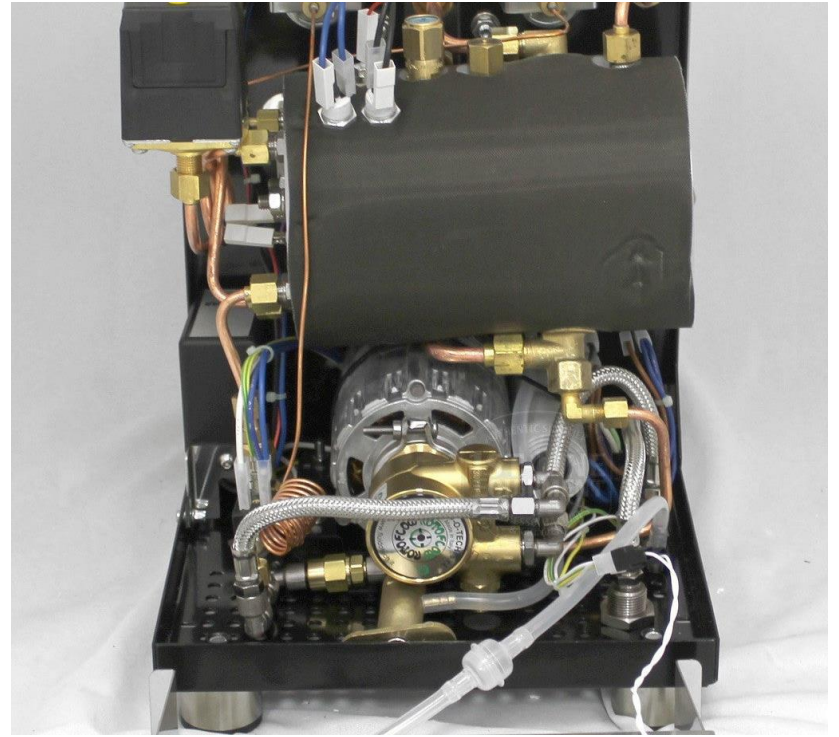
1. Suggest a database design in the ER model and convert to a relational database schema in a suitable normal form.
2. Write SQL queries, involving multiple relations, compound conditions, grouping, aggregation, and subqueries.
3. Use relational DBMSs from a conventional programming language in a secure manner.
4. Analyze/predict/improve query processing efficiency of the designed database using indices.
5. Reflect upon the evolution of the hardware and storage hierarchy and its impact on data management system design.
6. Discuss the pros and cons of different classes of data systems for modern analytics and data science applications.

Context

Lectures until now
(How to use a database)



Lectures from now
(How a database works)



Agenda

Architecture of a Database

Indexes (repeat)

- Full table scan vs. index scan
- Hash index vs. tree index
- Clustered vs. unclustered index
- Covering index

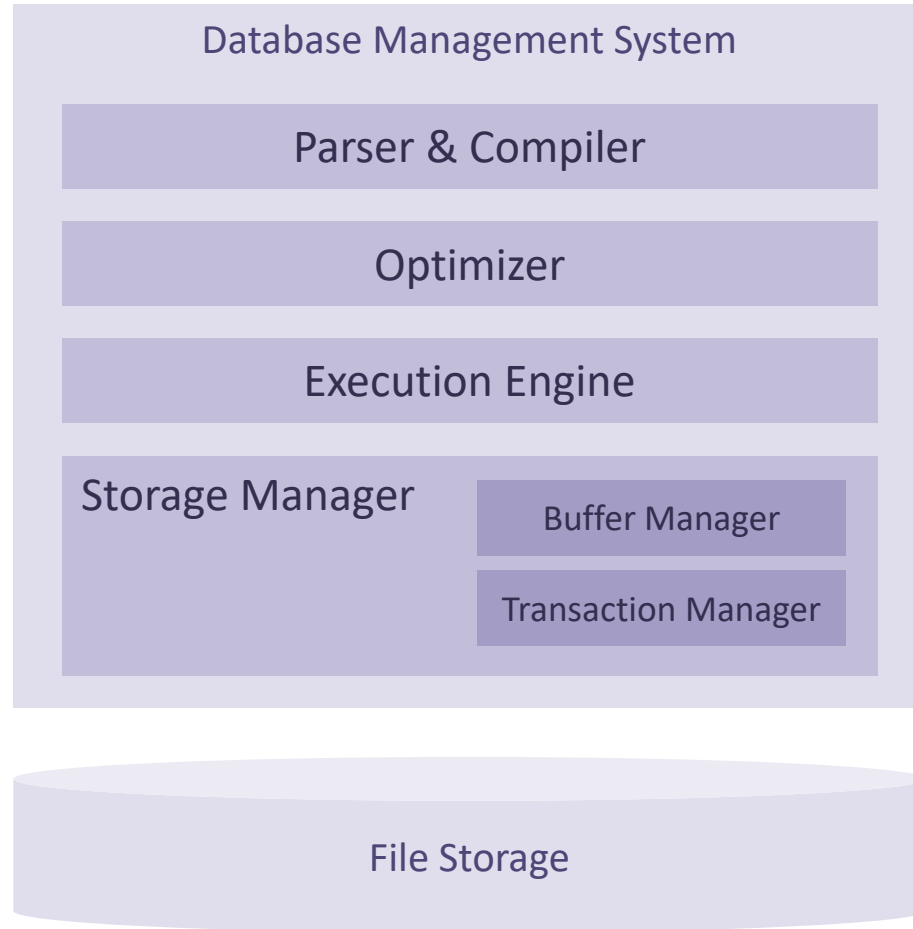
Min/Max Summarizes

Join Algorithms

- Nested loop join, hash join, sort-merge join

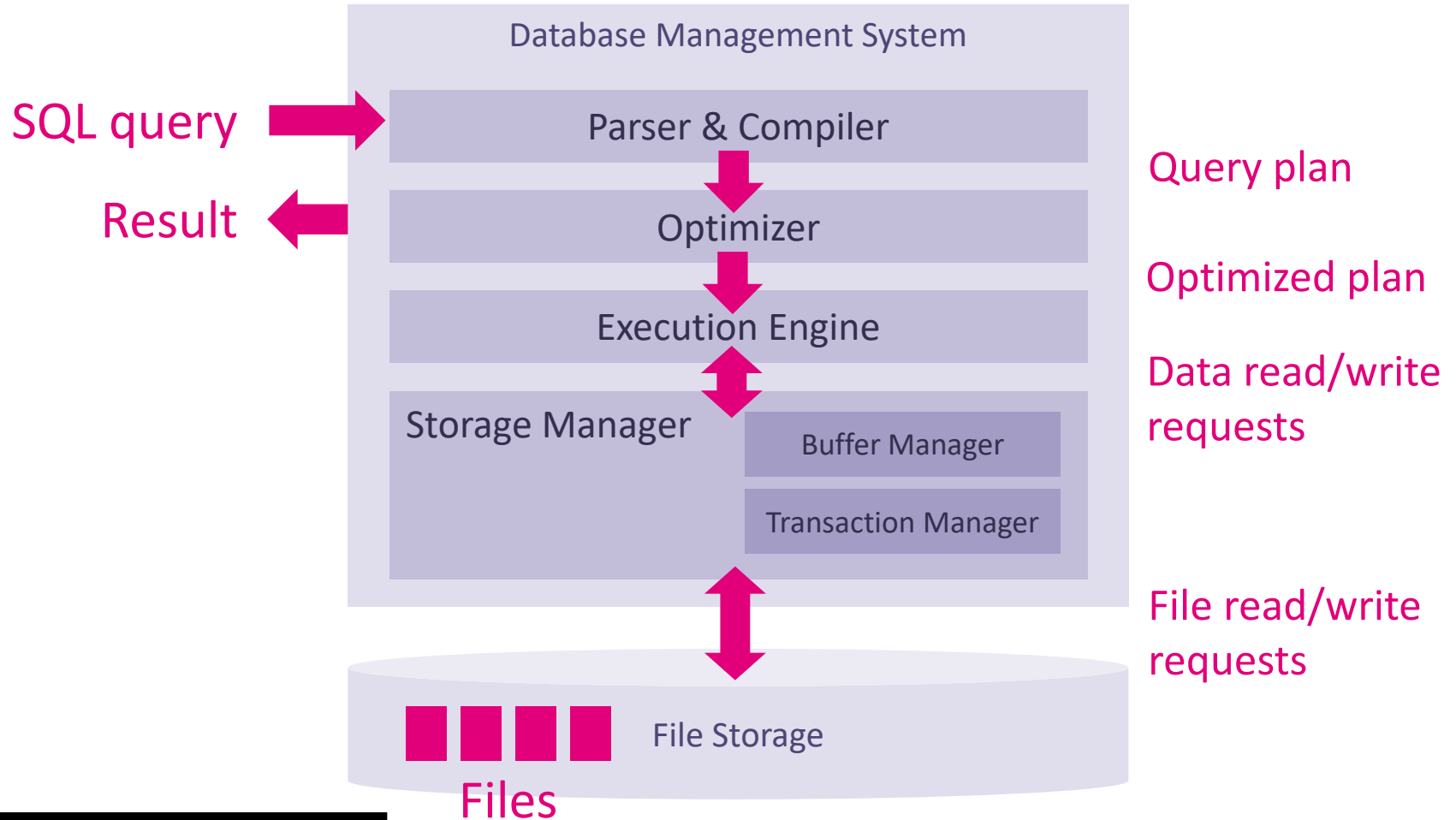
Architecture of a Database*

Important
components
of a database
system

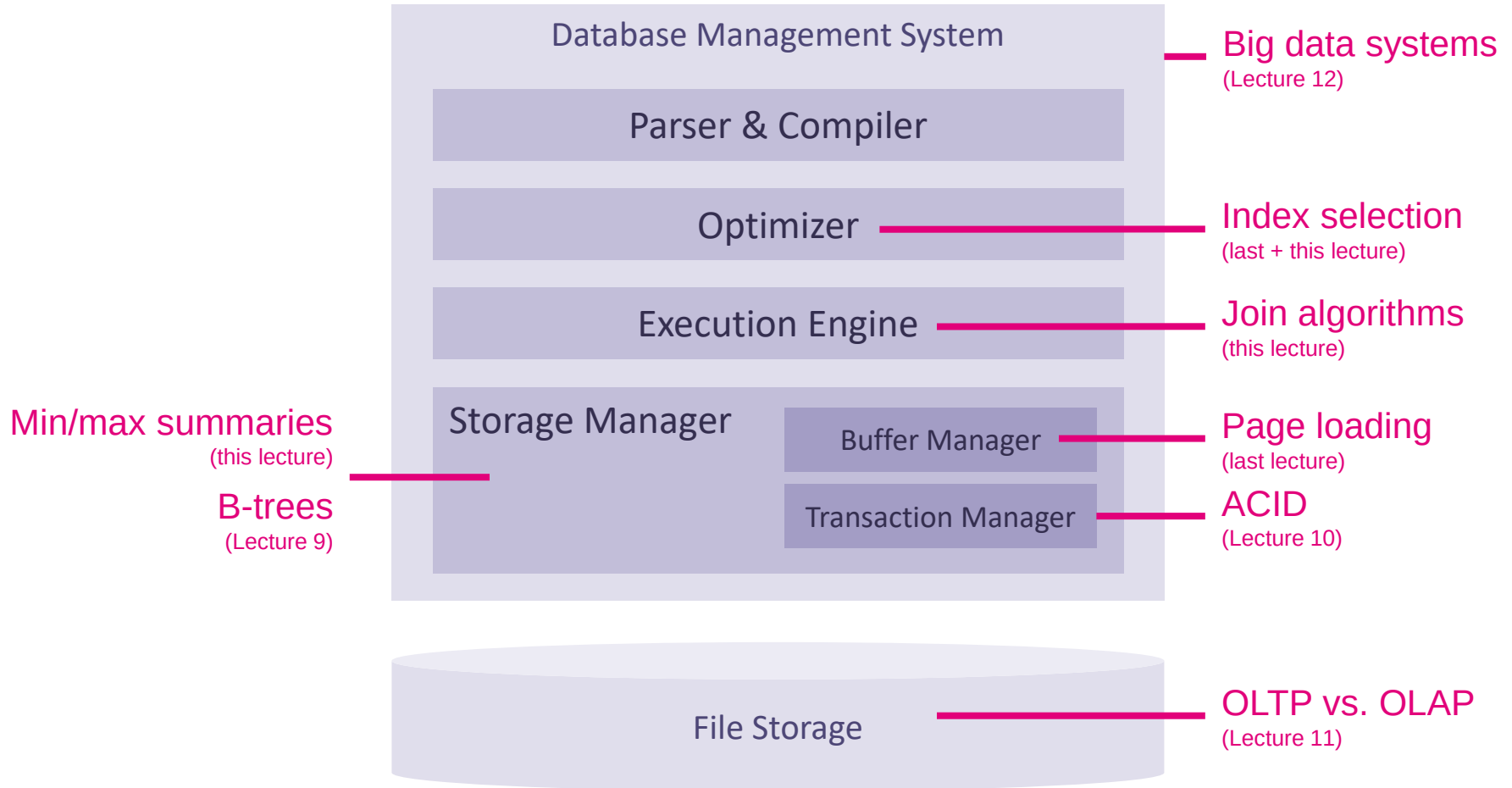


*Only a few important components shown. Many more components exist in practice, varying by database system.

Flow of Information



Components Covered in Rest of this Course



Agenda

Architecture of a Database

Indexes (repeat)

- Full table scan vs. index scan
- Hash index vs. tree index
- Clustered vs. unclustered index
- Covering index

Min/Max Summarizes

Join Algorithms

- Nested loop join, hash join, sort-merge join

Data in Files

Store data in files by tuples, one tuple after the next

- Header contains metadata like number of tuples, start byte of first tuple, checksums, and more
- Tuples can be of different lengths (for simplicity all same length below)

Product (ProductName, Price)

ProductName	Price
Small	85
Tea	100
Large	100
Filter	110
Fancy	125

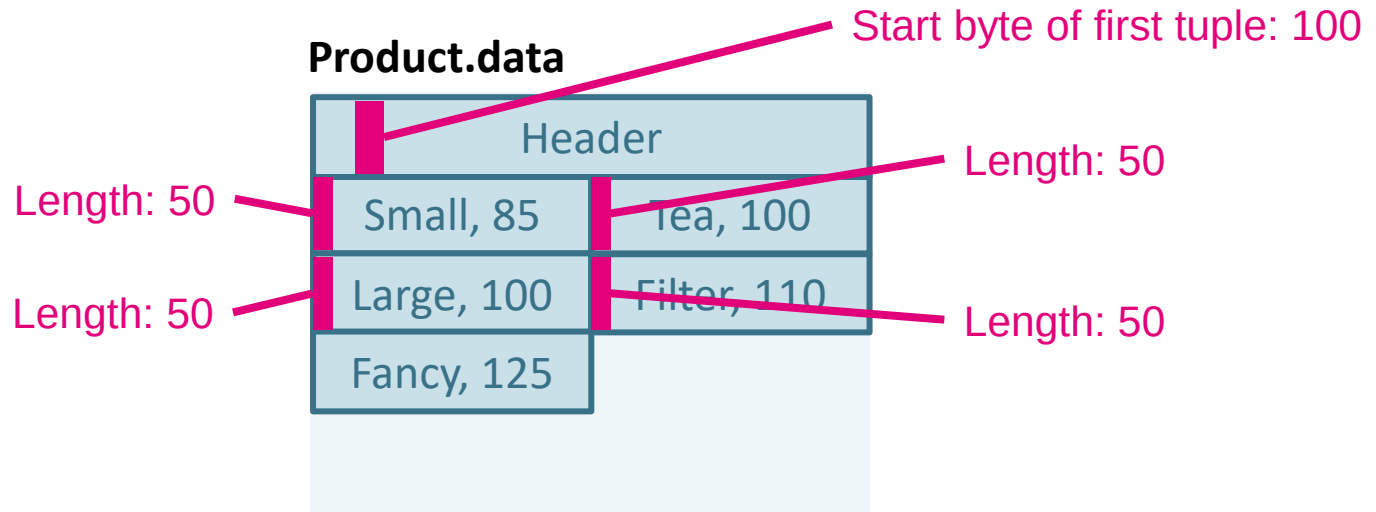


Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	

Files

- File is just an array of bytes
- Start position (start byte) of first tuple in header
- Length of tuple stored at the beginning of each tuple



Query Processing Example

Execution engine executes

```
select * from product
where productname = 'Fancy';
```

Steps:

- Read header of Product.data
- Jump to first tuple (byte = 100)
- Read tuple and compare name 'Small' to 'Fancy'
if match → add tuple to result set
- Jump to next tuple (byte = 150)
- ...repeat...

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	

Repeats for all tuples, in total 5 tuple reads and 5 comparisons

Full Table Scan

That was a “full table scan”

→ Can lead to bad performance if table gets large

Use an index to speed up processing



Manually, in SQL, when queries are slow:

```
create index productname_index  
on product(productname);
```

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	

Indexes Explained

An index is a way to “jump” to a tuple

- For example, the previous index could be implemented as a HashMap if the database was written in Java:

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	



```
// indexes product names to start bytes
private Map<String, Integer> createHashIndex() {
    Map<String, Integer> hashIndex = new HashMap<>();
    hashIndex.put("Small", 100);
    hashIndex.put("Tea", 150);
    hashIndex.put("Large", 200);
    hashIndex.put("Filter", 250);
    hashIndex.put("Fancy", 300);
    return hashIndex;
}
```

→ Stored in a file on disk typically

Example using the Index

Optimizer decides to use the index

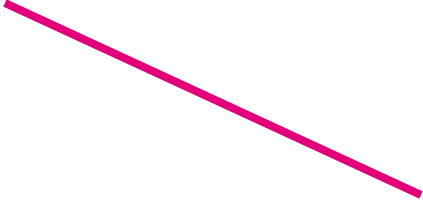
```
select * from product
where productname = 'Fancy';
```

Execution engine steps:

- Load `hashIndex` from file if not in memory
- Get start byte for 'Fancy': `hashIndex.get("Fancy")`;
- Jump to tuple at start byte 300 and add to result set

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	



1 index load (if not cached),
1 index lookup (the get call)
1 tuple read

Index Scan

That was an “index scan”

→ Usually very fast for point queries
(single-tuple lookups)

Hash index

- Works well for point queries and = comparisons

Tree index

- More general solution that works for range queries and <, > comparisons also
- Special data structure: B-trees (next lecture)

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	

Tree Index

Typically, trees are sorted, e.g. TreeMap in Java:

The map is sorted according to the natural ordering of its keys¹



```
// indexes product names to start bytes
private TreeMap<String, Integer> createTreeIndex() {
    TreeMap<String, Integer> treeIndex = new TreeMap<>();
    treeIndex.put("Small", 100);
    treeIndex.put("Tea", 150);
    treeIndex.put("Large", 200);
    treeIndex.put("Filter", 250);
    treeIndex.put("Fancy", 300);
    return treeIndex;
}
```


Example using the Tree Index

Optimizer decides to use `treeIndex`

```
select * from product
where productname >= 'Large'
and productname <= 'Tea';
```

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	

Execution engine steps:

- Load `treeIndex` from file if not in memory
- Call `treeIndex.subMap("Large", true, "Tea", true)`
- Jump to tuples at start bytes 200, 100, 150

Returns:

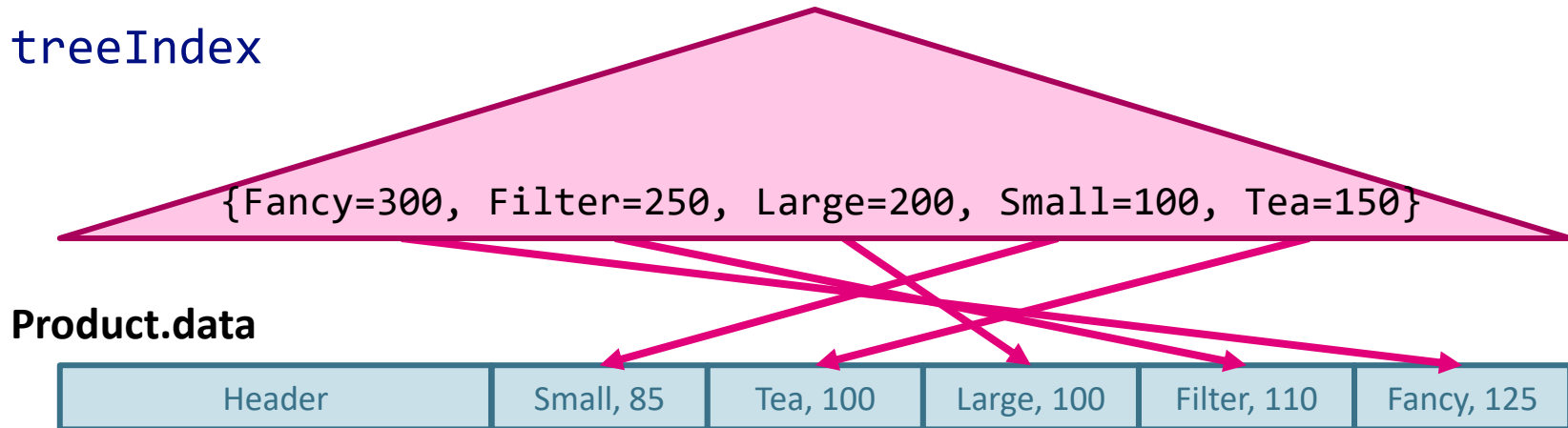
{Large=200, Small=100, Tea=150}

Unclustered Index

`treeIndex` is unclustered because order of index $\neq$ order of data in the file

- Execution engine jumped back and forth: 200, 100, 150
- Can lead to slower execution due to page faults
- Optimizer must be careful when deciding to use this index

`treeIndex`



Clustered Index

An index on price would be clustered because order of index = order of data in the file

- Read tuples sequentially, fewer page faults, faster performance

```
create index price_index on product(price);
```

Price index

{85=[100], 100=[150, 200], 110=[250], 125=[300]}

Product.data

Header	Small, 85	Tea, 100	Large, 100	Filter, 110	Fancy, 125
--------	-----------	----------	------------	-------------	------------

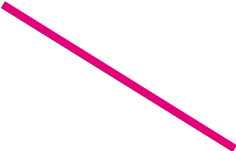
Implementation of Price Index in Java

Tree map supporting duplicate keys → list of values

```
// indexes prices to start bytes
private TreeMap<Integer, List<Integer>> createPriceIndex() {
    TreeMap<Integer, List<Integer>> priceIndex = new TreeMap<>();
    priceIndex.computeIfAbsent(85, _ -> new ArrayList<>()).add(100);
    priceIndex.computeIfAbsent(100, _ -> new ArrayList<>()).add(150);
    priceIndex.computeIfAbsent(100, _ -> new ArrayList<>()).add(200);
    priceIndex.computeIfAbsent(110, _ -> new ArrayList<>()).add(250);
    priceIndex.computeIfAbsent(125, _ -> new ArrayList<>()).add(300);
    return priceIndex;
}
```

Selecting products of the price range 100 to 110

```
priceIndex.subMap(100, true, 110, true)
```



Returns start bytes in sorted order:
{100=[150, 200], 110=[250]}

Covering Index

An index that includes all the columns a query needs, so the database engine can satisfy the query using only the index without having to access the actual table data.

Benefits:

- Improved performance because no I/O to table file

Drawbacks:

- Maintenance: increased overhead for write operations (INSERT, UPDATE, DELETE) because the index needs to be maintained for all columns it stores
- Memory consumption: additional columns stored in the index consume more space

Covering Index

`treeIndex` (index on product name) is already covering for queries that only return product names, e.g.:

```
select min(productName) from product;
```

If `treeIndex` also stored price attributes, the original query would not have to read from the data file:

```
select productname, price from product  
where productname >= 'Large'  
and productname <= 'Tea';
```

→ Whether an index is covering depends on the query and the data in the index.

Agenda

Architecture of a Database

Indexes (repeat)

- Full table scan vs. index scan
- Hash index vs. tree index
- Clustered vs. unclustered index
- Covering index

Min/Max Summarizes

Join Algorithms

- Nested loop join, hash join, sort-merge join

Min/Max Summaries

Indexes are beneficial, but also have maintenance overhead

- Adding a tuple to the product table requires to update the index
- Adding an index for every column, or combination of columns, is not feasible

Back to full table scans for many queries

- Optimize full table scans differently

Min/Max Summaries



also called: Min/max indexes
Zone maps
Block range indexes
Small materialized aggregates

File Structure

When data gets big, database may partition the data

- Groups of tuples/rows within a file
- Split data into multiple files

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25



partition into
groups of rows

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25

Row group 1

Row group 2

Row group 3

Collecting Statistics

- Partitioning happens during insertion of data already
- Possibility to collect statistics: min/max summaries and more
 - Stored e.g. in footer (Apache Parquet)

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25

Min/max summaries per
column for each row group

Name: Filter – Tea

Price: 85 – 110

Name: Cortado – Mocha

Price: 35 – 125

Name: Cola – Jasmin

Price: 25 – 30

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25
Footer	



Using Statistics During Compilation

Query optimizer decides which row groups to skip based on query predicate

- This is called “pruning”
- Usually happens at compile time, before executing the query

```
select * from product
where productname = 'Cola';
```

Predicate is
productname = 'Cola'

Optimizer skips row groups 1 and 2
because min(productname) of these
row groups > 'Cola'

Footer

Row group 1

Name: Filter – Tea

Price: 85 – 110

Row group 2

Name: Cortado – Mocha

Price: 35 – 125

Row group 3

Name: Cola – Jasmin

Price: 25 – 30

Query Plan after Optimization

```
select * from product
where productname = 'Cola';
```

“Query plan” before optimization

- Read all tuples of file Product.data
- If productname = ‘Cola’ → add to result
= 10 tuple reads + 10 comparisons

“Query plan” after optimization

- Read all tuples from row group 3
- If productname = ‘Cola’ → add to result
= 2 tuple reads + 2 comparisons

Footer

Row group 1

Name: Filter – Tea

Price: 85 – 110

Row group 2

Name: Cortado – Mocha

Price: 35 – 125

Row group 3

Name: Cola – Jasmin

Price: 25 – 30

Which row groups to read?

```
select * from product
where price >= 100;
```

Result: Row groups 1 and 2

Row group 3 pruned because $\max(\text{price}) < 100$

```
select * from product
where productname = 'Mocha'
and price < 60;
```

Result: Row group 2

Row group 3 pruned because $\max(\text{name}) < \text{'Mocha'}$

Row group 1 pruned because $\min(\text{price}) \geq 60$

```
select min(productName) from product;
```

Result: No row groups need to be read

Query can be answered using min/max statistics alone

Footer

Row group 1

Name: Filter – Tea

Price: 85 – 110

Row group 2

Name: Cortado – Mocha

Price: 35 – 125

Row group 3

Name: Cola – Jasmin

Price: 25 – 30

Min/Max Summaries of Files

Also works with files instead of (or in combination with) row groups

→ Min/max summaries per file (or per file and row group)

→ Min/max metadata stored in files or a key-value store

Product1.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110

Product2.data

Header	
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45

Product3.data

Header	
Jasmin, 30	Cola, 25

Minmax.metadata

Product1.data

Name: Filter – Tea
Price: 85 – 110

Product2.data

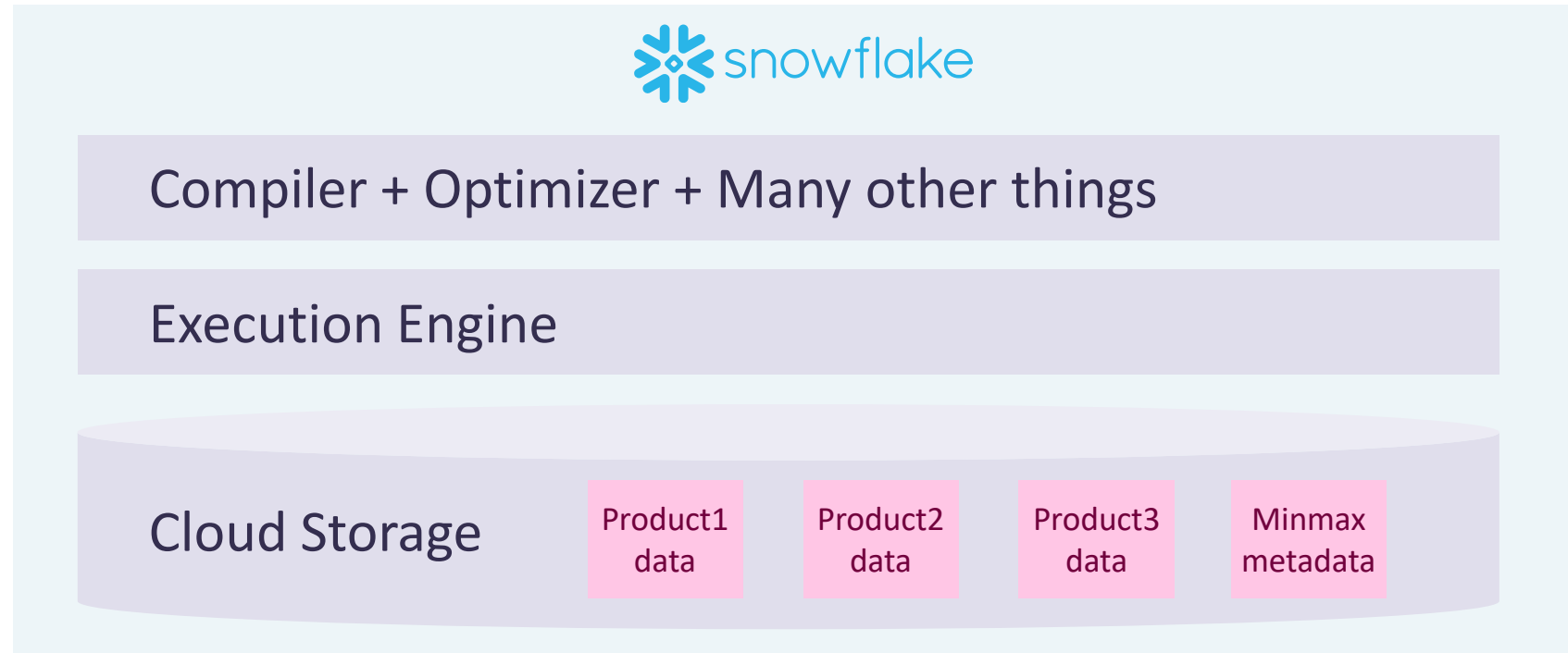
Name: Cortado – Mocha
Price: 35 – 125

Product3.data

Name: Cola – Jasmin
Price: 25 – 30

As a side note

If we move all files to cloud storage and add two layers of compute above, we have the architecture of Snowflake...



Summary of Min/Max Summaries 😊

During data insertion

- Partition data into row groups and/or files
- Collect min/max summaries per partition

During query compilation

- Prune partitions based on min/max summaries and query predicates → less data to read → faster execution
- Pruning also possible during execution (not covered today)

Agenda

Architecture of a Database

Indexes (repeat)

- Full table scan vs. index scan
- Hash index vs. tree index
- Clustered vs. unclustered index
- Covering index

Min/Max Summarizes

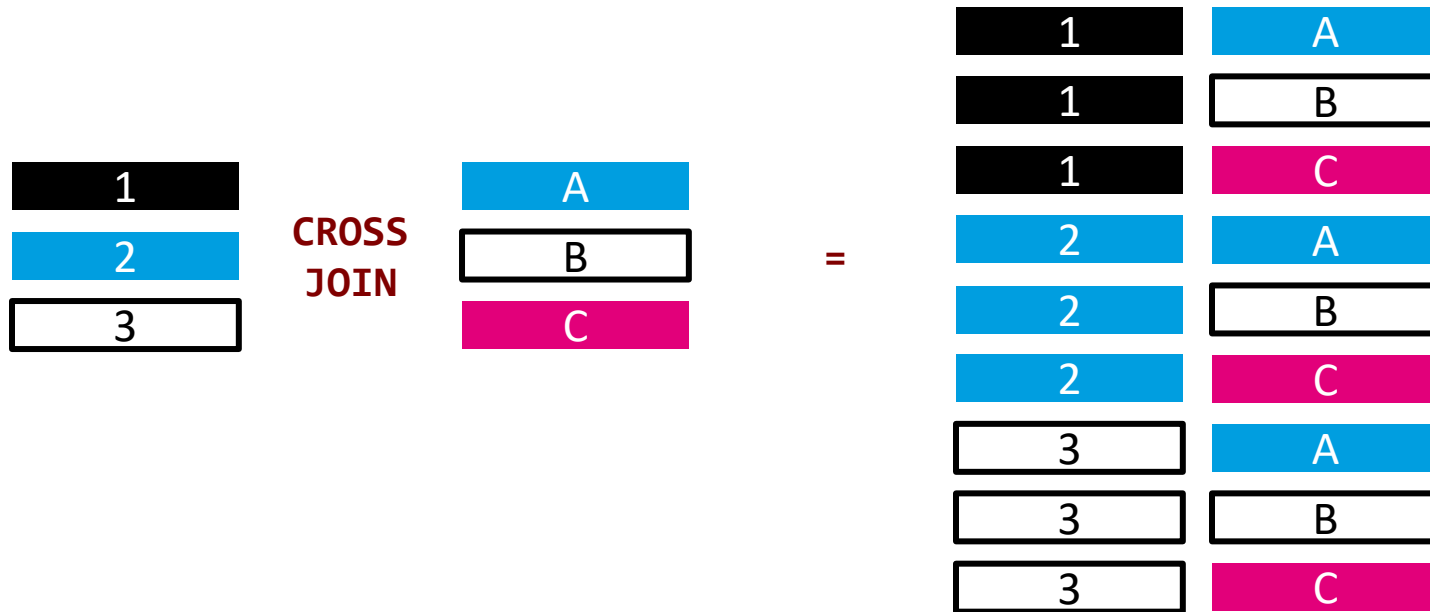
Join Algorithms

- Nested loop join, hash join, sort-merge join

*Job interview with one of the questions
that I always asked junior engineers
(won't be recorded)*

Joins (slide from Lecture 4)

First of all, let's look at **CROSS JOIN** first, because all other join types can be derived from **CROSS JOIN**¹.



Inner Join



How would you implement this in Java / Python / C++?

We will look at 3 different options.

Example

Purchase (PurchaseTime, ProductName, UserName)

PurchaseTime	ProductName	UserName
Feb 11, 9:55	Tea	Martin
Feb 12, 10:03	Small	Martin
Feb 12, 10:05	Small	Omar
Feb 12, 10:06	Large	Omar
Feb 19, 9:00	Small	Martin

Product (ProductName, Price)

ProductName	Price
Tea	100
Small	85
Large	100
Fancy	NULL

```
select *  
from purchase inner join product  
on purchase.productname = product.productname;
```

Java Classes / Records for Example

Records in Java holding purchase tuples, product tuples, and results.

- Hard-coded to make our lives simpler.

```
public record Purchase(  
    String time, String productName, String userName) {}  
  
public record Product(  
    String productName, int price) {}  
  
public record Result(  
    Purchase purchase, Product product) {}
```

Option 1: Nested Loop Join

Two nested loops: an outer loop & an inner loop.

- For every purchase row, iterate over every product row and check if the product names match

Purchase (PurchaseTime, ProductName, UserName)



PurchaseTime	ProductName	UserName
Feb 11, 9:55	Tea	Martin
Feb 12, 10:03	Small	Martin
Feb 12, 10:05	Small	Omar
Feb 12, 10:06	Large	Omar
Feb 19, 9:00	Small	Martin

Product (ProductName, Price)



ProductName	Price
Tea	100
Small	85
Large	100
Agency	NULL

5 * 4 comparisons → $O(N * M)$
~ quadratic runtime

Nested Loop Join Implementation

```
// nested loop join
public List<Result> joinNestedLoops(List<Purchase> purchases,
                                   List<Product> products) {

    List<Result> result = new ArrayList<>();

    for (Purchase purchase : purchases) {
        for (Product product : products) {
            if (purchase.productName().equals(product.productName())) {
                result.add(new Result(purchase, product));
            }
        }
    }

    return result;
}
```


Option 2: Hash Join

A hash table and a single loop.

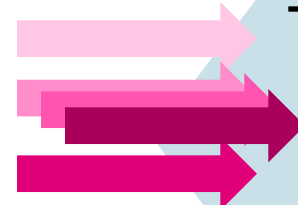
- Build a hash table for product rows using product names as keys. For each purchase row, look up the matching product row in the hash table.

Purchase (PurchaseTime, ProductName, UserName)



PurchaseTime	ProductName	UserName
Feb 11, 9:55	Tea	Martin
Feb 12, 10:03	Small	Martin
Feb 12, 10:05	Small	Omar
Feb 12, 10:06	Large	Omar
Feb 19, 9:00	Small	Martin

Product Hash Table



Tea	100
Small	85
Large	100
Fancy	NULL

4 hash table inserts & 5 hash look ups $\rightarrow O(M + N)$
~ linear runtime

Hash Join Implementation

```
// hash join
public List<Result> joinHash(List<Purchase> purchases,
                             List<Product> products) {

    List<Result> result = new ArrayList<>();

    Map<String, Product> productMap = new HashMap<>();
    for (Product product : products) {
        productMap.put(product.productName(), product);
    }

    for (Purchase purchase : purchases) {
        Product product = productMap.get(purchase.productName());
        if (product != null) {
            result.add(new Result(purchase, product));
        }
    }

    return result;
}
```

Option 3: Sort-Merge Join

Sort both tables, iterate over both together.

- Sort purchase rows and product rows by product name. Scan through both sorted lists, comparing product names.

Purchase (PurchaseTime, ProductName, UserName)



PurchaseTime	ProductName	UserName
Feb 12, 10:06	Large	Omar
Feb 12, 10:03	Small	Martin
Feb 12, 10:05	Small	Omar
Feb 19, 9:00	Small	Martin
Feb 11, 9:55	Tea	Martin

Product (ProductName, Price)



ProductName	Price
Fancy	NULL
Large	100
Small	85
Tea	100

2 table sorts & 2 table scans $\rightarrow O(N \log N + M \log M)$
~ log-linear runtime (sort dominates scan)

Sort-Merge Join Implementation

```
// sort-merge join
public List<Result> joinSortMerge(List<Purchase> purchases,
                                  List<Product> products) {
    List<Result> result = new ArrayList<>();

    purchases.sort(Comparator.comparing(Purchase::productName));
    products.sort(Comparator.comparing(Product::productName));

    int i = 0, j = 0;
    while (i < purchases.size() && j < products.size()) {
        Purchase purchase = purchases.get(i);
        Product product = products.get(j);
        int compare = purchase.productName().compareTo(product.productName());
        if (compare == 0) {
            result.add(new Result(purchase, product));
            i++; // note we only advance i, not j, because the next purchase
                // may match the same product
        } else if (compare < 0) {
            i++;
        } else {
            j++;
        }
    }
    return result;
}
```

Benchmarking the Java Implementation

```
// create benchmark data
int productCount = 1000;
int purchaseCount = 10_000_000;
List<Purchase> purchases = createPurchases(purchaseCount, productCount);
List<Product> products = createProducts(productCount);

// benchmark three join algorithms
benchmark("nested loop join", lecture8::joinNestedLoops, purchases, products);
benchmark("hash join", lecture8::joinHash, purchases, products);
benchmark("sort-merge join", lecture8::joinSortMerge, purchases, products);
```

Result:

nested loop join,	result size: 10000000,	time: 40524 ms
hash join,	result size: 10000000,	time: 628 ms
sort-merge join,	result size: 10000000,	time: 5074 ms

Benchmarking DuckDB

Export to CSV and import into DuckDB

```
~$ ls -lh *.csv
-rwxrwxrwx 1 mhent mhent 17K Mar 17 16:01 products.csv
-rwxrwxrwx 1 mhent mhent 374M Mar 17 16:01 purchases.csv
```

10 million purchases =
374MB of data

Run the join in DuckDB

```
D select count(*) from products join purchases on products.productName = purchases.productName;
```

count_star() int64

10000000

```
Run Time (s): real 0.051 user 0.561367 sys 0.000000
```

561 ms in DuckDB vs.
628 ms in my Java hash join implementation

Nested Loop Join

- Has to be two loops. If inner loop is replaced by an index look-up, it's not a nested loop join anymore.
- Only relevant for very small data when overhead of building hash table is comparatively large.

Hash Join

- Most common in practice! Other implementations are for corner cases (and for interviews 😊).
- Always index the smaller relation in the hash table.

Takeaways

Indexes avoid full table scans

→ By “jumping” to tuples

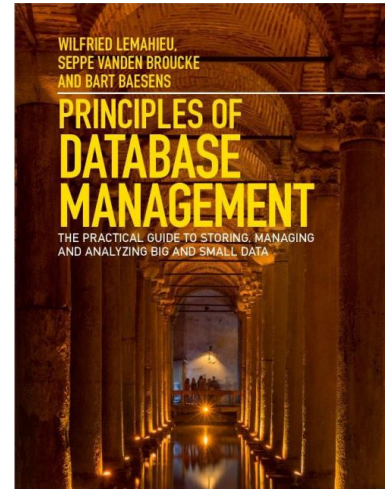
Min/max summaries avoid reading all data

→ By pruning partitions

Joins can be implemented differently

→ By being a clever engineer 😊

Readings: Chapter 2.1
& Chapter 13.1.4



Section 3.1 [Building a Data Management System for the Cloud: Lessons Learned and Future Directions](#)